中图分类号：TN384　　密　　级：公　开

济南大学

本科毕业设计



基于 Unity 的类恶魔城游戏的设计与实现

学　　院　信息科学与技术学院
专　　业　计算机科学与技术
班　　级　计科 2202
学　　生　葛广一
学　　号　202231222052
指导教师　赵亚欧

二〇二六年五月　日

济南大学毕业论文（设计）诚信承诺书

本人声明所呈交的毕业论文（设计）是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得济南大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

学生签名：_____　　日期：_____年_____月_____日

济南大学毕业论文（设计）使用授权书

本人完全了解济南大学有权保留本人所送交毕业论文（设计）的复印件和电子文档，可以采用影印、缩印或其他复制手段保存论文。本人电子文档的内容和纸质论文的内容相一致。除在保密期内的保密论文外，允许论文被查阅和借阅，可以公布（包括刊登）论文的全部或部分内容。的公布（包括刊登）授权济南大学教务处

办理。

学生签名：_____　　导师签名：

日期：_____年_____月_____日　　日期：_____年_____月_____日

摘　要

本文围绕 Unity 平台下的 2D 横版动作游戏 Demo 展开，主要目标是完成一款类恶魔城风格的小型可运行作品，项目使用 C#脚本进行功能编写，并通过有限状态机组织玩家、普通敌人和 Boss 的行为流程；角色移动、跳跃、冲刺、普通攻击和技能释放，主要依靠 Rigidbody2D、碰撞检测以及 Animator 之间的配合来完成，战斗部分包含生命值管理、伤害判定、受击反馈、飞剑、黑洞和 Boss 二阶段激光等内容，关卡部分加入了落石机关、骷髅王和 Tilemap 通路事件，玩家会先经历基础战斗，再过渡到最终 Boss 战，系统还配有开始、暂停、死亡、通关菜单以及血条、音效和 Windows 打包支持，测试情况说明，该 Demo 能够完成开始、探索、战斗、机关躲避、Boss 挑战和结算这一整条流程，可以作为毕业设计展示和论文分析的实践基础。

关键词：Unity；C#；2D 动作游戏；有限状态机；Boss 战

ABSTRACT

This thesis takes a 2D Metroidvania-style action game demo as the development object. The project is built in Unity and written in C#. Finite state machines are used for the player, common enemies and the boss, so each action stage has a clearer script boundary. Rigidbody2D, Collider2D, Animator and Tilemap work with custom scripts to handle movement, jumping, dashing, attacks, skills and terrain collision. The combat part keeps health management, damage detection, knockback feedback, sword throwing, black hole control and a two-phase boss fight with laser warnings. The demo also adds falling stone traps, an enhanced Skeleton King enemy and a Tilemap gate opened after the Skeleton King is defeated. Together with menus, health bars, sound effects and Windows build support, the project forms a complete playable process for graduation demonstration.

Key words: Unity; C#; 2D action game; finite state machine; boss fight

目　　录

摘　　要.....I
ABSTRACT.....II
1 绪论.....1
1.1 选题背景与意义.....1
1.2 国内外研究现状.....1
1.3 本文主要工作.....2
1.4 论文组织结构.....2
2 需求分析.....3
2.1 系统目标.....3
2.2 功能需求分析.....3
2.2.1 玩家控制需求.....4
2.2.2 战斗与技能需求.....4
2.2.3 敌人与 Boss 需求.....4
2.2.4 UI 与流程需求.....4
2.2.5 关卡机关与流程解锁需求.....5
2.3 非功能需求分析.....5
2.4 可行性分析.....6
3 系统总体设计.....7
3.1 开发环境与技术选型.....7
3.2 系统总体架构.....7
3.3 核心模块划分.....8
3.4 状态机设计.....9
3.5 资源与场景组织设计.....10
3.6 关卡流程与交互事件设计.....11
4 系统详细设计与实现.....13
4.1 玩家控制系统实现.....13
4.2 角色属性与伤害判定实现.....15
4.3 技能系统实现.....16
4.4 普通敌人 AI 实现.....18
4.5 关卡机关与精英敌人实现.....19
4.6 Boss 战斗系统实现.....21
4.7 UI 与音效系统实现.....24
5 系统测试.....26

5.1 测试环境与测试方法.....26
5.2 功能测试.....27
5.3 典型问题与优化.....28
5.4 测试结果分析.....30
结　　论.....32
参 考 文 献.....35
致　　谢.....36

1 绪论

1.1 选题背景与意义

随着数字娱乐产业持续发展，独立游戏也给 2D 横版动作游戏留下了较大的创作空间，和大型 3D 游戏相比，2D 游戏在美术资源、场景规模和运行成本上相对更容易控制，不过操作手感、碰撞反馈、敌人行为和关卡节奏并不会因为画面维度减少而变得简单；类恶魔城游戏通常会包含横版探索、动作战斗、能力成长和 Boss 挑战，玩家的体验既来自角色本身的操作，也来自敌人反馈、地图结构和流程安排，基于 Unity 完成一款规模不大但流程相对完整的 Demo，既能展示游戏开发中的综合能力，也比较适合作为计算机专业毕业设计实践。

从软件工程角度看，本课题不是简单把素材摆到场景里，也不是只完成一个角色移动脚本就结束，输入响应、物理控制、动画切换、敌人 AI、技能系统、UI 交互和资源管理都要放进同一套稳定流程中；动作游戏里的状态变化比较频繁，如果结构没有提前理清，代码中容易出现条件交叉、动画混乱和后续扩展困难等问题，有限状态机把角色行为拆成若干阶段，移动、攻击、受击和死亡分别处理，代码之间的耦合会低一些，阅读、排查和调试也会更顺。

课题实践以 Unity 2D 项目为载体，围绕“能运行、能演示，也能把实现过程说明白”的毕业 Demo 目标来展开，项目通过玩家控制、普通敌人、Boss 战、技能、

血量 UI 和菜单流程构成基本玩法闭环，为论文中的需求分析、总体设计、详细实现和测试验证提供了工程依据；通过这一项目训练 C#面向对象编程、Unity 组件化开发、状态机建模、动画事件调用和用户交互设计等能力，能够让毕业设计不只停留在文字说明上，而是有一个可以实际运行的作品作为支撑。

1.2 国内外研究现状

就游戏工业而言，Unity 已经成为 2D 和 3D 游戏开发中应用较广的一类引擎，它提供 Tilemap、Animator、Rigidbody2D、Collider2D、AudioSource、Canvas 等成熟组件，可以覆盖横版动作游戏从关卡构建到角色控制、从动画播放到 UI 展示的主要需求；不少独立游戏会借助 Unity 的组件化体系快速搭建原型，再通过脚本对输入、物理参数和状态切换进行调整，用来获得更贴近目标玩法的操作体验。

在游戏 AI 设计中，有限状态机、行为树和分层状态机都是较常见的组织方式，拿毕业设计规模的 2D 动作游戏 Demo 来说，有限状态机的实现成本较低，结构也比较直观，调试时更容易看到对象当前处在哪个阶段；普通敌人可以用待机、巡逻、追踪、攻击、受击和死亡等状态描述行为，Boss 可以在此基础上加入传送、施法、二阶段转场等状态，用来表现更复杂的战斗节奏；和把全部逻辑堆在一个 Update 函数中的方式相比，状态机能让每个状态只处理本阶段要做的事情，后续维护也会轻一些。

在 2D 角色控制的研究和实践中，所谓“手感”往往和输入响应速度、移动加速度、跳跃高度、碰撞检测稳定性、攻击前后摇以及受击反馈都有关系，本项目没有追求完整商业作品的复杂度，而是把重点放在毕业 Demo 能稳定展示的核心体验上，即玩家能较顺畅地移动、跳跃、冲刺和攻击，敌人能在一定范围内发现玩家并作出反应，Boss 能给出阶段变化和清楚的攻击提示；这样的取舍有利于在有限开发周期内形成稳定、可答辩的系统成果。

1.3 本文主要工作

本文主要围绕基于 Unity 的 2D 类恶魔城动作游戏 Demo 进行设计与实现，完成的工作主要体现在几个部分。

一是完成玩家控制系统设计，项目基于 Unity 输入、Rigidbody2D、Collider2D 和 Animator 实现玩家移动、跳跃、冲刺、墙面交互、普通攻击、飞剑技能和黑洞技能，并把这些动作放到玩家状态机中统一管理，使状态切换有比较清楚的边界。

二是完成敌人与 Boss 战斗系统设计，普通敌人以骷髅敌人为代表，能够完成巡逻、发现玩家、追踪、攻击、受击和死亡等行为，Boss 以 DeathBringer 为核心，包含近战攻击、法术攻击、传送、二阶段激光转场和死亡反馈等流程。

三是完成战斗判定与角色属性系统设计，项目实现 CharacterStats、PlayerStats、EnemyStats 等脚本，用来管理最大生命值、当前生命值、伤害、无敌时间和死亡状

态，攻击判定则通过碰撞检测、触发器和动画事件之间的配合来完成。

四是完成 Demo 交互与发布支持，系统包含玩家血条、敌人血条、Boss 血条、开始菜单、暂停菜单、死亡菜单、通关菜单和基础音效管理，同时完成 Windows 可运行程序打包，便于脱离 Unity 编辑器进行现场演示。

五是完成关卡机关与流程解锁设计，项目在主线路径中加入落石机关、骷髅王精英敌人和击败后打开通道的 Tilemap 门，玩家在进入最终 Boss 区域前会经历探索、躲避、战斗和解锁等环节，使 Demo 的流程完整度更高一些。

1.4 论文组织结构

在具体内容安排上，本文共分为六个主要部分；第 1 章为绪论，介绍课题背景、研究现状、选题意义和主要工作，第 2 章进行需求分析，说明系统目标、功能需求、非功能需求和可行性，第 3 章进行系统总体设计，介绍开发环境、架构设计、模块划分、状态机思路和资源组织方式，第 4 章说明玩家控制、技能、敌人 AI、Boss 战和 UI 系统的具体实现，第 5 章给出系统测试方案和运行结果，结论部分则对本设计已经完成的内容、存在不足以及后续可扩展方向进行总结。

2 需求分析

2.1 系统目标

本系统的目标是实现一款能够独立运行并进行现场演示的 2D 类恶魔城动作游戏 Demo，Demo 并不追求覆盖完整商业游戏的全部内容，而是优先保证核心玩法闭环能够走通，也就是玩家进入游戏后能控制角色完成基础探索，与普通敌人发生战斗，并在最终挑战 Boss 后获得通关结果；系统需要在有限开发周期内保持功能边界清楚、运行状态稳定、结构也方便说明，这样才能支撑毕业论文撰写和答辩演示。

结合毕业设计任务要求和当前项目实际情况，系统目标可以概括为三点：一是实现响应较好的玩家控制和战斗反馈，二是实现基于有限状态机的普通敌人与 Boss 行为逻辑，三是实现菜单、血量显示、音效和打包发布等 Demo 支撑功能，这些目标共同构成论文分析和作品演示的主要依据。

2.2 功能需求分析

按照游戏 Demo 的运行流程来看，系统功能可以划分为玩家控制、战斗技能、敌人 AI、关卡机关、Boss 战、UI 交互和音效发布等模块，各模块需求如表 2.1 所示。

表 2.1 系统主要功能需求

模块	功能需求	说明
玩家控制	移动、跳跃、冲刺、墙面交互、攻击	保证角色能够完成横版动作游戏的基础操作

技能系统	Dash、Sword、Blackhole	飞剑支持投掷、插入、召回和接剑，黑洞支持范围控制与持续伤害
普通敌人	巡逻、发现、追踪、攻击、受击、死亡	通过状态机组织骷髅敌人的行为流程
Boss 战	近战、施法、传送、二阶段激光、死亡	提供阶段变化和较完整的战斗演示
UI 系统	开始、暂停、死亡、通关、血条	保证游戏流程可开始、暂停、重开和结束
音效与发布	攻击、受击、死亡、菜单等音效，Windows 打包	增强反馈并支持脱离编辑器运行
关卡机关	落石陷阱、精英怪、击败后开门	通过场景事件提高探索与战斗之间的衔接

2.2.1 玩家控制需求

玩家角色是系统交互的核心，角色需要能根据键盘输入完成左右移动、跳跃、冲刺、墙面滑落、墙跳和普通攻击，并在动画状态上表现出对应动作；为了减少输入和动画之间互相干扰，玩家行为不适合全部堆在一个脚本中，而要通过状态机拆分为待机、移动、跳跃、空中、冲刺、攻击、瞄准飞剑、接剑等状态。

玩家控制还要具备基本的容错性和反馈性，就攻击过程而言，攻击期间要产生相应的攻击判定，受击后要扣除生命值并触发反馈，死亡后要进入死亡表现并弹出死亡菜单；飞剑和黑洞技能需要能由玩家输入触发，并且要和玩家状态切换保持一致，避免技能释放时破坏原有动作流程。

2.2.2 战斗与技能需求

战斗系统既要支持玩家通过普通攻击、飞剑和黑洞对敌人造成伤害，也要支持

敌人和 Boss 对玩家造成伤害，伤害不应只表现为数值变化，还要联动受击反馈、击退方向、无敌时间、死亡判定和音效播放；对于毕业 Demo 来说，战斗系统的重点在于判定要清楚、反馈要看得出来、流程也要尽量稳定。

技能部分中，飞剑技能需要具备瞄准轨迹、投掷、命中插入、召回和接剑等流程，黑洞技能需要具备生成、范围显示、冻结敌人、持续造成多次伤害和结束销毁等流程；技能实现尽量复用统一的 Skill 基类和 SkillManager 管理方式，把不同技能之间的耦合控制在较低程度。

2.2.3 敌人与 Boss 需求

普通敌人在未发现玩家时要进行基础巡逻或等待，发现玩家后进入追踪或攻击状态，当玩家脱离检测范围后，敌人应在一定时间后退出战斗状态，减少全图追踪或坐标重叠抖动等不自然现象；敌人受击后还要根据攻击来源产生正确方向的击退，并在生命值归零时播放死亡表现。

Boss 战是 Demo 中最主要的展示内容，Boss 需要有比普通敌人更复杂的状态切换，包括靠近玩家、近战攻击、施法攻击、传送、二阶段转场和死亡；二阶段转场要有较明显的提示，就激光攻击而言，可以通过瞄准线和延迟发射给玩家留下反应时间，玩家也能通过冲刺进行躲避，这样战斗节奏和演示效果都会更清楚。

2.2.4 UI 与流程需求

游戏流程需要从开始菜单进入游戏，运行过程中按 Esc 能够暂停，并给出继续、重新开始和退出选项，玩家死亡后要暂停游戏并显示重新开始或退出按钮，Boss 被击败后也要显示通关界面；Demo 要有明确的开始和结束，血条 UI 要能随角色生命值变化实时更新，Boss 血条也要方便玩家观察战斗进程。

就当前 Demo 而言，音效系统需要为攻击、受击、死亡、技能、Boss 激光和菜单点击等操作提供基础反馈，音效虽然不是系统核心算法，但会明显影响演示时的完成度和操作反馈感，所以将其作为毕业 Demo 的辅助功能进行实现。

2.2.5 关卡机关与流程解锁需求

为了让 Demo 不是从普通敌人直接跳到 Boss 战，关卡中需要加入少量能够解释清楚的环境机关，这类机关不追求复杂解谜，而是承担节奏调节和演示扩展作用；玩家在移动过程中要观察场景、躲避落石，并在击败精英敌人后打开后续道路，使角色控制、敌人战斗和场景交互能够出现在同一条演示路径中。

落石机关在玩家进入检测区域后触发，每个石块都要单独判断，不能因为一个触发器而带动整排石块一起下落；下落阶段设置加速度和较高速度，用来制造需要冲刺躲避的压力，回收阶段速度较慢，机关能够反复使用，石块碰到玩家时直接触发死亡，使危险结果比较明确。

精英骷髅王用于连接机关区域和后续通路，它可以复用普通骷髅敌人的状态机

和攻击逻辑，但通过更高生命值、更大体型和头顶光环与普通敌人区分；击败骷髅王后，场景右侧柱子底部的通路应解除阻挡，玩家能够继续前进，该需求没有引入完整钥匙背包系统，而是通过敌人死亡事件直接控制 Tilemap 通路，比较符合当前 Demo 范围。

需求边界在这一阶段也需要写清楚。类恶魔城游戏常会继续加入地图回环、能力门槛、道具成长和存档系统，但这些内容会把开发重心拉向更大的关卡框架。当前作品把目标收束到一条可以完整演示的路线，先让玩家完成移动和战斗，再经过机关与精英敌人，最后进入 Boss 区域。这样处理并不等同于减少系统设计，而是把需求集中到已经能在 Demo 中验证的交互链上，使功能说明和实际运行结果保持一致。

从验收角度看，功能是否存在并不能只靠脚本文件数量判断，还要观察它能否在一局流程中被玩家自然用到。开始菜单要能进入游戏，角色要能到达机关区域，落石要对冲刺躲避形成压力，骷髅王死亡后通路要确实打开，Boss 战还要给出失败或通关反馈。只要其中一个节点中断，后面的演示就会被截断，所以本设计把流程连通性当成需求分析中的重要约束，而不是把各模块孤立列出。

2.3 非功能需求分析

系统非功能需求大致包括运行稳定、便于维护、现场能演示，并且后续还能继

续增加内容；稳定性方面，Demo 运行时不应出现阻塞流程的空引用异常、角色卡死、菜单无法响应或 Boss 无法结束等问题，可维护性方面，玩家、敌人、Boss、技能、属性和 UI 应尽量分布在独立脚本与目录中，便于后续查找和修改。

可演示性方面，项目需要能在 Windows 环境下打包为可运行程序，这样答辩或展示时就不依赖 Unity 编辑器；可扩展性方面，虽然本阶段没有实现完整背包、复杂存档和大型非线性地图，但系统应保留继续添加新技能、新敌人、新关卡和存档功能的结构基础。

2.4 可行性分析

技术层面上，Unity 2022.3 LTS 已经提供较完整的 2D 开发支持，Tilemap 能够搭建关卡，Rigidbody2D 和 Collider2D 能够处理移动与碰撞，Animator 用于控制序列帧动画，Canvas 负责菜单和血条 UI，C#支持面向对象封装，适合编写 Entity、CharacterStats、Skill 和 State 等基础类。

从时间安排上看，项目把范围控制在毕业 Demo 而不是完整商业游戏，优先完成玩家控制、普通敌人、Boss 战和基础 UI 等价值较高的功能，以此降低开发风险；从成本上看，项目开发主要依赖 Unity 个人版、公开课程资源和本地开发环境，不需要额外服务器和商业部署成本，所以本课题在毕业设计周期内具有较好的实现可行性。

3 系统总体设计

3.1 开发环境与技术选型

本设计使用 Unity 引擎和 C#语言开发，主要开发环境与工具如表 3.1 所示[8-9,15]。

表 3.1 开发环境与技术选型

类别	工具或技术	用途
游戏引擎	Unity 2022.3 LTS	场景搭建、物理系统、动画、UI 和打包发布
开发语言	C#	编写玩家、敌人、技能、UI 和管理器脚本
物理组件	Rigidbody2D、Collider2D	处理角色移动、碰撞和触发检测
关卡工具	Tilemap	绘制横版地形、背景和碰撞区域
动画系统	Animator、Animation Clip	管理角色动作、攻击和死亡动画
版本管理	Git、GitHub	保存项目版本并支持回退与同步
发布平台	Windows	生成可直接运行的 Demo 程序

在技术选择上，Unity 能够以组件化方式组织 GameObject 及其脚本，适合毕业设计中快速搭建可运行作品，C#脚本则负责描述各模块行为，就玩家状态切换、敌人检测、Boss 攻击、技能生成、血量更新和菜单响应而言，都可以通过脚本与 Unity 组件配合完成；和从底层图形库开始实现相比，使用 Unity 能让开发重点更多

放在玩法逻辑和软件结构上，更符合本课题目标[8-9]。

3.2 系统总体架构

从运行过程看，系统可以理解为由“场景对象层、组件行为层、状态控制层、关卡事件层和界面反馈层”共同组成，场景对象层承载 Tilemap、玩家、敌人、Boss、摄像机和 Canvas 等可见对象，组件行为层通过 MonoBehaviour 脚本把输入、物理、动画、碰撞和音频连接起来，状态控制层负责把角色在不同阶段的行为拆成明确状态，界面反馈层承担血量、菜单和通关提示等展示任务，这样的划分使项目虽然规模不大，但仍然保持了较清楚的软件结构。

在模块协作上，玩家输入不会直接修改所有对象状态，而是先驱动玩家状态机，再由状态机调用移动、攻击或技能相关方法；敌人和 Boss 也不是在 Update 中堆叠大量条件判断，而是通过各自状态类判断追踪、攻击、受击和死亡等行为，UI 模块则通过角色属性事件或流程控制脚本被动更新，减少界面逻辑反向侵入战斗逻辑，这样可以降低脚本之间的强依赖，也方便在答辩演示中说明系统运行链路。

系统整体采用分层和模块化设计思路，底层由 Unity 场景、Tilemap、动画、碰撞体、音频资源和预制体构成，脚本层按照功能划分为玩家控制、敌人 AI、Boss 逻辑、技能系统、角色属性、UI 系统和音效管理，运行层再通过 Unity 生命周期函数和动画事件驱动状态更新、碰撞判定和界面刷新。

在对象组织上，玩家、普通敌人和 Boss 会继承或复用 Entity、CharacterStats 等基础能力，玩家和敌人的复杂行为由状态机管理，每个状态负责自身进入、更新和退出逻辑；技能模块通过 SkillManager 统一获取技能组件，减少玩家脚本直接管理所有技能细节，UI 模块监听角色生命值变化或游戏流程事件，并负责显示菜单和血条。

各模块之间并不是完全隔离运行。玩家输入进入状态机后，会调用 Entity 中的速度、碰撞和受击能力，战斗结果再落到 CharacterStats 中的生命值变化；血量发生变化时，UI 脚本通过事件刷新显示，若生命值归零，还会继续影响死亡菜单、通关菜单或关卡门状态。也就是说，项目表面上表现为角色移动、敌人追踪和界面变化，底层实际存在一条由输入、状态、判定、属性到反馈的执行链，这条链路也是总体架构需要说明的重点。

项目中的对象创建方式也做了区分。飞剑属于需要反复复用并配置碰撞、刚体与控制器的攻击对象，因此以 Prefab 为主；Boss 法术同样依赖预制体保存表现和判定。瞄准点和部分黑洞表现则允许在运行时创建，在资源未额外配置时，脚本仍能生成可见对象并完成技能流程。两种方式各有用途，Prefab 适合保存较稳定的对象结构，运行时生成更适合轻量提示和临时效果，这种混合方式也减少了 Demo 对大量手工场景配置的依赖。

3.3 核心模块划分

结合当前项目代码结构，系统主要模块划分如表 3.2 所示。

表 3.2 系统核心模块划分

模块	代表脚本或目录	主要职责
玩家模块	Scripts/Player	玩家状态机、移动、跳跃、冲刺、攻击、技能触发
敌人模块	Scripts/Enemy/Skeleton	普通敌人的巡逻、追踪、攻击、受击和死亡
Boss 模块	Scripts/Enemy/DeathBringer	Boss 状态切换、传送、施法、二阶段激光和死亡
技能模块	Scripts/Skills	Dash、Sword、Blackhole 等技能创建、冷却和控制
属性模块	Scripts/Stats	生命值、伤害、无敌时间和死亡判定
UI 模块	Scripts/UI	血条、开始菜单、暂停菜单、死亡菜单和通关菜单
管理模块	Scripts/Managers	音效播放、玩家引用和全局辅助管理
机关模块	Scripts/Traps	落石触发、石块回收、骷髅王死亡后 Tilemap 通路解锁

这样的模块划分使论文中的详细实现可以按照功能展开，玩家控制系统可以围绕 PlayerStateMachine 和各类 PlayerState 进行说明，敌人 AI 可以围绕 EnemyStateMachine 和 Skeleton 状态进行说明，关卡机关可以围绕 FallingStoneTrap 和 SkeletonKingTileGate 进行说明，Boss 战可以围绕 DeathBringer 各状态与激光控制器进行说明；各模块之间通过明确引用和事件调用协作，减少单个脚本承担过多职责的情况。

3.4 状态机设计

本设计中状态机的重点是让每个状态只处理当前阶段最核心的输入与行为，拿玩家控制来说，待机状态负责检测移动、跳跃和攻击输入，移动状态负责水平速度和移动动画，空中状态负责下落、落地和墙面检测，冲刺状态则在短时间内屏蔽普通移动并施加冲刺速度；通过这种方式，玩家同一时间只处在一个主要动作状态中，能够减少多种输入同时触发时产生的冲突。

敌人与 Boss 状态机不只处理动画切换，还承担 AI 决策节奏控制，普通敌人的战斗状态会根据玩家位置、检测结果和 battleTime 决定继续追踪还是退出战斗，Boss 战斗状态则加入攻击冷却、法术释放、传送和二阶段判断；状态机并不是为了追求复杂 AI 算法，而是服务于稳定、可控、可解释的目标，使 Demo 能够表现出较完整的战斗流程。

有限状态机是本系统比较重要的结构之一，它的基本思想是把一个对象运行时的行为拆分为若干互斥或阶段性状态，并在满足条件时从当前状态切换到目标状态；每个状态通常包含 Enter、Update 和 Exit 等逻辑，其中 Enter 用来初始化状态，Update 用来持续检测输入或环境条件，Exit 用来恢复或清理状态。

玩家状态机更多是用来解决动作频繁切换的问题，玩家可能处于待机、移动、跳跃、空中、冲刺、普通攻击、飞剑瞄准、接剑和黑洞释放等状态，如果这些逻辑全部写在一个 Update 函数中，不同输入之间容易互相影响；通过状态机拆分后，攻

击状态只处理攻击期间的行为，冲刺状态只处理冲刺速度和结束条件，接剑状态只处理接剑动画和后续切换，代码含义会更清楚。

敌人和 Boss 状态机主要用于控制 AI 行为，普通敌人根据检测范围、玩家位置、攻击距离和战斗时间在巡逻、追踪、攻击、受击与死亡之间切换，Boss 在普通敌人基础上加入施法、传送和二阶段转场；二阶段转场通过激光预警和延迟发射制造节奏变化，使 Boss 战不只是单一追逐和近战攻击。

状态切换还要和 Unity 本身的执行方式配合。持续检测更适合放在 Update 中，例如玩家是否按下移动键、敌人是否还处于追踪时间内；具有明确时点的行为则交给动画事件或协程处理，例如攻击动画某一帧生成判定，落石经过预警后进入下落，Boss 激光在锁定目标后延迟发射。把这些职责分开后，动作表现和逻辑时机更容易对齐，也能减少每个状态承担过多计时细节。

状态机并不负责项目中的全部工作。若把伤害计算、音效加载、血条更新和关卡开门都塞进角色状态类，状态数量虽然清楚，类之间仍会重新变乱。本设计把状态机更多用于组织行为阶段，把属性变化交给 CharacterStats，把通用受击和速度接口放在 Entity，把界面流程交给 UI 脚本，把音效播放交给 AudioManager。这样的边界更接近当前 Demo 需要的结构，也便于在论文中分别说明。

3.5 资源与场景组织设计

资源组织上，项目按照功能和用途进行划分，Player 目录放置玩家脚本，Enemy 目录放置普通敌人和 Boss 脚本，Skills 目录保存技能脚本，Stats 和 UI 目录分别处理属性和界面，动画控制器、图片、音效和预制体也按类型整理；当飞剑技能出现空引用时，可以从 SkillManager、Sword_Skill 和飞剑 Prefab 三个位置逐步排查。

场景组织方面，Demo 采用线性演示流程，玩家出生后先接触基础移动和普通敌人，再通过落石机关和骷髅王区域，最后进入 Boss 战区域；这样安排并不是为了构建完整大型地图，而是为了保证答辩现场能在较短时间内展示玩家控制、敌人 AI、技能释放、Boss 阶段切换和菜单流程，场景中的碰撞层级也作了区分，地面和墙体用于阻挡角色，背景层只承担视觉表现，从而减少背景装饰对物理逻辑的干扰。

就当前 Demo 而言，项目资源主要放置在 Assets 目录下，脚本、动画、图像、预制体、音效和场景分别按照用途组织；Scripts 目录按照 Player、Enemy、Skills、Stats、UI 和 Managers 等子目录拆分，Prefabs 目录用来保存飞剑、Boss 法术、敌人和 UI 相关预制体等可复用对象，Resources/Audio/SFX 目录则用于存放运行时由 AudioManager 加载的音效资源。

场景设计以 SampleScene 为主，地面、平台和背景层级由 Tilemap 搭建，玩家、普通敌人、Boss、相机和 UI Canvas 在同一场景中协作；为了保证演示流程简洁，场景重点放在基础探索、普通敌人战斗、落石机关、精英敌人和 Boss 区域，当前阶段没有扩展大型地图和完整存档流程，这样能够降低调试压力，也能让论文内容和实

际项目保持一致。

场景组织还要考虑对象层级与可编辑性。地面、墙体和平台由 Tilemap 完成，背景层只承担视觉展示，玩家、普通敌人、骷髅王和 Boss 则作为可单独配置的场景对象放置；机关脚本挂到对应石块或关卡对象上，参数可以在 Inspector 中调整。这样做后，修改落石间距、Boss 触发距离或骷髅王位置时，不必改动整套地图逻辑，场景编辑和脚本维护之间仍有清楚分工。

镜头与场景边界同样会影响关卡展示。当前 Demo 使用相机跟随方式让玩家保持在画面关注区域内，平台高度、落石位置和 Boss 区域距离都需要结合可见范围观察，若镜头过低，画面会露出不合适的背景边界；若 Boss 放得过近，玩家还在前段流程时就可能进入最终战斗。由此可见，场景设计不仅是摆放素材，还要让相机、碰撞、敌人检测和演示路线彼此配合。

后续章节将在总体设计基础上继续说明各模块的具体实现，内容包括玩家状态机、技能控制器、敌人检测逻辑、Boss 二阶段激光、血量 UI 和菜单流程等部分。

3.6 关卡流程与交互事件设计

在总体设计上，关卡流程采用“基础操作展示—普通敌人战斗—机关躲避—精英敌人解锁—Boss 战”的线性结构，这一结构能够在较短演示时间内覆盖项目的大部分核心模块，也能让玩家逐步接触不同类型的交互；如果直接把玩家放到 Boss 面

前，虽然可以展示 Boss 战，但玩家控制、普通敌人 AI 和场景机关等内容不容易自然呈现，加入中间区域后，Demo 过程会更完整。

关卡事件没有单独建立复杂任务系统，当前做法是使用 Unity 场景对象和脚本事件，玩家进入落石下方时，传感器调用对应石块的 Trigger 方法，石块下落、停留和回收结束后，会重新进入可触发状态；骷髅王死亡时，CharacterStats 触发 onHealthChanged 事件，机关门脚本检测到生命值归零后移除指定 Tilemap 格子，原本被柱子挡住的位置随后就可以通行。

这种设计的好处是实现成本较低，并且与现有角色属性和碰撞系统保持一致，机关区域更多依赖 Collider2D、Trigger 和 Coroutine，解锁区域更多依赖 Tilemap 和角色死亡事件，不需要额外引入背包、任务、存档等较重系统；对于毕业 Demo 而言，这种做法既能表现关卡互动，也能控制开发风险。

表 3.3 关卡流程与触发事件设计

流程阶段	触发条件	系统反馈
出生与基础移动	玩家点击开始游戏	恢复时间流动，玩家进入待机或移动状态
普通敌人区域	玩家靠近或攻击骷髅敌人	敌人进入战斗状态，刷新 battleTime 并追踪玩家
落石机关区域	玩家进入石块正下方传感器	对应石块预警、加速下落、造成即死判定并回收
骷髅王区域	玩家接近或攻击精英骷髅	骷髅王以更高血量和体型进行战斗
通路解锁	骷髅王生命值归零	指定 Tilemap 砖块被移除，柱子底部变为可通行
Boss 区域	玩家进入 Boss 战范围	Boss 启动战斗流程并进入阶段判断

关卡流程还要考虑演示顺序和玩家学习成本，玩家第一次进入 Demo 时，如果

立即面对 Boss，容易因为不熟悉移动、冲刺和攻击节奏而失败，也难以表现出普通敌人和技能系统的作用；所以关卡在 Boss 前安排普通骷髅、落石机关和骷髅王，实际上是在逐步引导玩家熟悉操作，普通骷髅用来验证攻击和受击反馈，落石机关要求玩家使用移动与冲刺，骷髅王则把基础战斗提高到小型精英战，这种顺序不需要额外文字教学，玩家通过场景内容本身就能理解下一步目标。

从工程实现角度看，关卡事件层被控制在较轻量的范围内，项目没有引入完整任务系统、事件总线或存档驱动的关卡状态，而是利用现有 CharacterStats 事件、Collider2D 触发器和 Tilemap 接口完成局部事件；这种选择和毕业 Demo 目标相匹配，一方面能展示“击败敌人后改变场景”的流程逻辑，另一方面不会因为系统过大导致调试成本上升，如果后续要扩展为正式游戏，可以在此基础上抽象出 GameEvent 或 LevelEvent 管理器，把落石、门锁、钥匙和检查点统一纳入事件管理。

关卡中的可通行区域和不可通行区域主要由 TilemapCollider2D 承担碰撞职责，这样地图编辑与碰撞生成能够保持一致，绘制砖块时就能形成阻挡，移除砖块时也能同步解除碰撞；骷髅王通路解锁正是利用这一点实现，脚本不直接移动墙体，也不单独关闭某个 BoxCollider2D，而是移除 Ground Tilemap 中的具体 cell，由于视觉和物理来源一致，玩家看到砖块消失后就能通过，减少“看起来能走但被挡住”或“看起来有墙但能穿过”的不一致问题。

在关卡节奏上，落石机关与骷髅王形成了两类不同压力，落石机关主要考验玩

家对空间和时间的判断，玩家只要观察石块位置并适时冲刺就能通过，骷髅王则更偏向战斗操作，需要多次攻击、躲避并观察血条；两种内容都复用已有系统，却带来不同体验，落石复用碰撞和生命值，骷髅王复用敌人状态机和血条，这种复用方式能在较少新增代码的情况下增加流程层次，也是小规模 Demo 扩展内容时比较稳妥的做法。

4 系统详细设计与实现

4.1 玩家控制系统实现

玩家移动的实现依赖 Rigidbody2D 和自定义速度设置方法，水平移动时，脚本根据输入方向设置刚体横向速度，并通过动画参数反映移动状态，跳跃时脚本会调整竖直方向速度，使角色能够脱离地面，冲刺状态则在短时间内覆盖普通移动逻辑，让角色快速向指定方向移动；由于这些行为都会改变刚体速度，项目将速度设置集中到 Entity 给出的方法中，减少不同状态对刚体的重复操作。

墙面交互是横版动作游戏中比较容易出错的部分，本设计通过 GroundCheck 和 WallCheck 等检测点判断玩家是否接触地面或墙体，并在墙面滑落和墙跳状态中限制或改变速度，避免玩家贴墙时仍然按普通空中状态处理；这样可以形成更符合横版动作游戏习惯的墙跳体验，虽然 Demo 没有实现复杂能力解锁，但墙面交互已经能展示角色控制系统的扩展能力。

输入屏蔽机制看起来不大，却会直接影响演示体验，开始菜单被点击后，鼠标输入可能继续传到游戏中，如果不做处理，玩家进入场景时可能直接打出一次普通攻击；项目通过 BlockActionInputFor 短暂屏蔽攻击和技能输入，把菜单点击和战斗输入分开，Demo 开始阶段也会显得更自然。

玩家控制系统是本设计的基础模块，Player 脚本继承自 Entity，在 Awake 方法中

创建 PlayerStateMachine，并依次实例化 idleState、moveState、jumpState、airState、dashState、wallSlideState、wallJumpState、primaryAttackState、aimSwordState、catchSwordState 和 blackholeState 等状态；Start 方法中调用 stateMachine.Initialize(idleState)，使玩家默认从待机状态进入游戏，运行时 Player.Update 在非死亡且游戏未暂停的情况下调用当前状态的 Update 方法，从而把不同动作逻辑交给对应状态类处理。

这种写法的主要优点是把玩家行为拆成多个相对独立的状态，待机状态主要检测输入并切换到移动、跳跃或攻击，移动状态处理水平速度与动画参数，跳跃和空中状态处理竖直速度、落地检测和墙面检测，冲刺状态则根据 dashSpeed、dashDuration 和 dashDir 短时间改变玩家速度；不同状态之间通过 stateMachine.ChangeState 方法切换，减少在一个脚本中写大量互相嵌套的输入判断。

玩家动作还要与动画和技能释放配合，项目通过 PlayerAnimationTriggers 接收动画事件，在普通攻击动画的关键帧执行攻击检测，在飞剑动作中调用 SkillManager.instance.sword.CreateSword 生成飞剑；玩家接剑时调用 CatchTheSword 方法，先将当前状态切换到 catchSwordState，再销毁场景中的飞剑对象并播放接剑音效，由于动画事件与状态机相互配合，系统能够让技能释放、攻击判定和动画表现基本同步。

Player 脚本中还加入 BlockActionInputFor 和 IsActionInputBlocked 方法，用来在

开始菜单点击后短时间屏蔽攻击输入，减少玩家点击开始游戏时立即触发普通攻击；Die 方法负责设置死亡标记、播放死亡音效、触发死亡动画并冻结刚体，玩家死亡后不再继续响应移动和攻击输入，整体来看，玩家控制模块围绕状态机、动画事件和刚体控制三部分展开，能够满足毕业 Demo 中移动、战斗和技能释放的需求。

玩家控制实现中，状态和物理并不是两套互不相干的逻辑。状态类决定当前应该接受哪些输入，Entity 提供 SetVelocity 与 ZeroVelocity 等公共接口，再由 Rigidbody2D 呈现位移结果。比如冲刺阶段需要短时间覆盖普通移动，墙面检测会阻止不合适的冲刺触发，死亡后刚体被冻结并停止继续响应输入。这样一来，角色动作变化虽然较多，实际执行入口仍然比较集中，后续调节速度、冲刺时间或墙面交互时也能沿着明确位置排查。

普通攻击与技能释放还需要避免只看输入而忽略表现节奏。项目把部分判定放到动画关键帧上，玩家看到攻击动作挥出时，攻击区域才会真正检测敌人；飞剑则先进入瞄准状态，再在投掷事件触发后生成对象。菜单按钮点击后，系统会使用未受 Time.timeScale 影响的时间短暂屏蔽动作输入，这个处理专门解决开始游戏时鼠标点击被误当成攻击的问题。对动作游戏 Demo 而言，这类细节不显眼，却会直接影响操作是否顺手。

（此处替换为游戏运行截图：玩家移动、跳跃、冲刺或普通攻击画面）

图 4.1 玩家控制与基础场景运行效果

表 4.1 玩家主要状态及职责

状态名称	动画参数	主要职责
Idle / Move	Idle、Move	处理待机、水平移动和基础输入检测
Jump / Air	Jump	处理跳跃、空中下落、落地和墙面检测
Dash	Dash	根据方向和持续时间完成冲刺动作
Attack	Attack	播放普通攻击动画并通过动画事件产生伤害判定
AimSword / CatchSword	AimSword、CatchSword	控制飞剑瞄准、投掷后的接剑流程
Blackhole	Jump	释放黑洞技能并等待技能控制器结束

4.2 角色属性与伤害判定实现

属性系统使用事件驱动方式连接战斗数据与界面显示，CharacterStats 负责维护生命值和伤害数值，角色受到伤害后会更新 currentHealth，并通过 onHealthChanged 通知血条刷新；血条脚本只关心显示比例，不直接参与伤害计算，攻击脚本只负责触发伤害，不直接修改 UI，这样的职责划分能让数据变化和界面反馈保持同步，同时减少模块耦合。

受击方向的处理对动作反馈影响较大，如果只按照敌人自身朝向击退，当玩家从敌人背后攻击时，敌人可能反而向玩家方向移动，表现会比较不自然；所以项目在伤害方法中支持传入攻击来源位置，并根据攻击者与受击者的相对位置计算击退方向，这样普通攻击、飞剑和 Boss 攻击在反馈上更统一，也更接近玩家的直觉。

为了减少角色在短时间内连续受到多次伤害，属性系统还保留无敌时间和死亡状态判断，角色死亡后会设置 isDead，后续伤害和行为更新不再重复触发死亡逻辑；这一机制在 Boss 激光、黑洞持续伤害和近战攻击同时存在时较为重要，能够减少同一帧或短时间内重复扣血、重复播放死亡动画等问题。

角色属性系统主要由 CharacterStats、PlayerStats 和 EnemyStats 构成，CharacterStats 中定义 maxHealth、currentHealth、damage、isDead 和 isInvincible 等字段，TakeDamage 方法负责扣除生命值，并通过 onHealthChanged 事件通知 UI 刷新；当 currentHealth 减少到 0 时，CharacterStats 会调用 Die 方法记录死亡状态，PlayerStats 和 EnemyStats 在此基础上分别服务于玩家和敌人，使不同角色能够拥有独立血量与伤害数值。

Entity 类承担角色通用行为，包含刚体、动画、碰撞检测和受击处理等基础能力，Damage 方法给出多个重载，既可以根据 Transform 获得攻击来源，也可以直接传入 Vector2 位置和伤害值；这样设计后，普通攻击、飞剑、黑洞、Boss 法术和激光都能复用统一的受击入口，为了让击退方向更自然，项目根据伤害来源位置计算受击方向，而不是简单按照敌人自身朝向后退，从而减少玩家从背后攻击时敌人反向靠近玩家的问题。

伤害判定主要通过 Collider2D、Trigger 和动画事件完成，玩家普通攻击会在攻击动画关键帧检测 AttackCheck 区域内的敌人，并调用

enemy.Damage(player.transform)，飞剑命中敌人或地面后停止移动，插入目标并关闭碰撞，玩家再次按键时飞剑返回；Boss 法术和二阶段激光则通过各自控制器检测玩家碰撞并调用 player.Damage，这种做法能让伤害来源和受击对象之间保持清楚关系，也方便后续扩展新的攻击形式。

血量 UI 通过订阅 CharacterStats.onHealthChanged 事件刷新 Fill Image 的 fillAmount，事件驱动方式减少 UI 在每一帧主动查询角色生命值，也能降低不必要的耦合；当玩家死亡或 Boss 死亡后，UI 流程脚本会根据生命值状态弹出对应菜单，使角色属性系统不仅影响战斗结果，也参与整个游戏流程控制。

受击流程在项目中并不是简单减去一个数值。伤害来源先进入 Entity.Damage，再由 CharacterStats 判断生命值和死亡状态，随后触发受击闪烁、击退与声音反馈；如果目标已经死亡，后续受击流程就不应继续制造重复动作。普通敌人、玩家和 Boss 虽然外观与行为不同，但这条公共流程可以复用，所以攻击来源增加后，系统不需要为每一种伤害重新写一套死亡和血量处理逻辑。

击退方向的改动也能反映战斗反馈的设计思路。项目根据攻击来源位置判断水平受力方向，玩家站在敌人左侧攻击时，敌人会向右后退，站在右侧攻击时则向左后退；当两者横坐标几乎重合时，再使用朝向作为兜底。与单纯按照敌人背后方向退开相比，这种做法更符合玩家观察到的碰撞关系，也避免背刺时敌人反而靠向玩家。

表 4.2 伤害相关脚本职责

脚本	职责	说明
CharacterStats	维护生命值、伤害和死亡状态	提供 TakeDamage、IncreaseHealthBy 和 MakeInvincible 等方法
Entity	提供通用受击、击退和检测能力	玩家、普通敌人与 Boss 共享基础逻辑
PlayerAnimationTriggers	在攻击动画中触发伤害检测	保证攻击判定与动画帧同步
Sword_Skill_Controller	处理飞剑命中、插入和返回	命中敌人后调用 Enemy.Damage
DeathBringerPhaseLaser	处理二阶段激光伤害	预警结束后检测激光范围内玩家

4.3 技能系统实现

就当前 Demo 而言，飞剑技能的实现体现了技能生命周期管理思路，飞剑生成前由 Sword_Skill 计算鼠标方向并显示轨迹点，飞剑生成后由 Sword_Skill_Controller 接管运动、命中、插入、召回和销毁流程；玩家对象通过 AssignNewSword 保存当前飞剑引用，当飞剑尚未召回时再次按下右键会触发召回，而不是重复生成新的飞剑，这样能够减少场景中出现多个失控飞剑的情况。

飞剑命中后的处理需要兼顾视觉和碰撞，飞剑接触敌人或地面后会停止刚体速度，关闭碰撞体并挂接到目标或停留在命中位置，使其表现为插入状态，召回时再重新移动到玩家附近，并在接近玩家时切换到接剑状态；这种流程比单纯发射后立即销毁更能表现技能过程，也便于答辩时说明对象状态变化。

黑洞技能采用范围控制和定时伤害结合的方式实现，黑洞生成后根据最大尺寸

逐渐放大，并检测范围内敌人，进入黑洞影响范围的敌人会被临时冻结或限制行动，在持续时间内受到多次伤害；项目没有直接把敌人持续吸到中心，而是使用冻结处理，减少敌人与玩家重叠时出现左右抽搐，这体现了玩法效果与物理稳定性之间的折中。

技能系统由 Skill 基类、SkillManager 以及具体技能脚本组成，SkillManager 通过 GetComponent 获取 Dash_Skill、Sword_Skill 和 Blackhole_Skill 实例，玩家状态只需要通过 SkillManager 访问技能，不需要直接了解技能内部细节；这种结构把输入状态和技能生成逻辑分离，能够降低玩家脚本的复杂度。

飞剑技能由 Sword_Skill 和 Sword_Skill_Controller 共同实现，Sword_Skill 负责读取鼠标右键输入，通过 Camera.main.ScreenToWorldPoint 将鼠标屏幕坐标转换为世界坐标，再计算玩家到鼠标的方向作为发射方向；瞄准过程中，GenerateDots 生成轨迹点，DotsPosition 根据初速度、重力和时间计算抛物线位置，玩家在释放前可以看到飞剑的大致轨迹，释放时 CreateSword 实例化飞剑预制体，并调用 SetupSword 传入方向、重力、玩家引用和返回速度。

飞剑控制器负责飞剑发射后的状态变化，飞剑初始阶段根据 SetupSword 中的方向设置速度，命中敌人或地面后进入插入状态，停止刚体运动并关闭碰撞，以减少持续触发伤害；当玩家再次按右键召回飞剑时，控制器让飞剑朝玩家位置移动，飞剑接近玩家后调用 player.CatchTheSword 切换到接剑状态并销毁飞剑对象，使飞剑技

能不只是一次性投射物，而是具备投掷、命中、插入、召回和接剑的完整交互。

黑洞技能由 Blackhole_Skill 和 Blackhole_Skill_Controller 实现，黑洞生成后，控制器根据 maxSize 和 growSpeed 逐渐放大自身范围，并在持续时间内按 damageInterval 对进入范围的敌人造成多次伤害；敌人进入黑洞触发器后会被加入 enemies 列表，并调用 enemy.FreezeTime(true)冻结行动，离开或黑洞结束时再通过 ReleaseEnemies 恢复敌人状态，为了减少敌人在黑洞中心与玩家重叠时不断改变朝向，黑洞逻辑选择冻结敌人而不是持续吸向中心，这样整体效果会更稳定。

飞剑技能的瞄准过程还体现了可视化反馈的重要性。Sword_Skill 先把鼠标屏幕坐标转换到世界坐标，再根据玩家位置计算发射方向，轨迹点位置则叠加初速度和重力项，显示出近似抛物线。轨迹点不是最终命中承诺，而是给玩家一个释放前参考，所以当鼠标位置变化时会同步刷新。该设计让投掷技能比固定方向子弹更容易理解，也让答辩演示时能够直观看到输入和技能方向之间的关系。

飞剑发射后仍然需要受控生命周期。Player 对象只保存当前飞剑引用，场景里已有飞剑时不会再次生成新的飞剑；飞剑命中敌人或地面后会停止运动并进入插入状态，召回阶段再向玩家移动，接近玩家后切换到接剑状态并销毁对象。这个流程把投射物从一次性伤害对象变成了可回收技能，也避免重复实例化造成画面和判定混乱。

黑洞技能采用另一种处理思路。控制器会记录进入范围的 Enemy 列表，并按持

续时间计算伤害间隔，设定次数没有消耗完时就逐次调用 TakeBlackholeDamage；范围结束、敌人离开或对象销毁时，控制器都会释放被冻结的敌人，避免技能结束后 AI 仍停在冻结状态。这里没有继续追求强吸附效果，是因为项目测试中发现敌人与玩家重叠时容易出现方向抖动，改成范围冻结和多段伤害后，技能作用仍然清楚，运行结果也更稳定。

技能冷却在本项目中按交互结果计算，而不是在按键触发后统一开始计时。飞剑投出后仍可能停留在敌人或地面上，只有玩家将其召回并接住，冷却才会启动；黑洞则在范围效果结束后等待玩家重新落地，再进入冷却。这样处理后，技能动作、状态切换和界面提示能够对齐，技能处于冷却期时也不会先播放上升动作再被脚本拦回。冷却显示由 UI_SkillCooldownPanel 读取技能剩余比例，在屏幕右上角通过图标和径向遮罩提示飞剑、黑洞是否可再次使用。

黑洞多段伤害还补充了死亡边界处理。控制器只有在实际命中敌人后才消耗一次伤害次数，当范围内目标仍在时，会等待既定次数完成后再结束技能；若最后一段伤害正好击杀敌人，循环会先保存当前 Enemy 引用，再继续处理冻结与释放，避免碰撞体关闭后敌人列表变化引发索引异常。被冻结目标进入死亡流程时还要恢复 Animator 速度，否则死亡参数虽然已经写入，动画也可能停留在冻结帧。

（此处替换为游戏运行截图：飞剑轨迹、飞剑命中或黑洞技能效果）

图 4.2 技能释放运行效果

表 4.3 技能实现流程

技能	触发方式	核心流程
Dash	LeftShift	检测冷却与方向，进入 Dash 状态并短时间改变速度
Sword	鼠标右键	显示轨迹点、投掷飞剑、命中插入、召回、接剑销毁
Blackhole	技能输入	生成黑洞、扩大范围、冻结敌人、造成多次伤害、缩小销毁

4.4 普通敌人 AI 实现

就当前 Demo 而言，普通敌人的检测逻辑更多依赖前方射线、距离判断和战斗计时，当敌人看到玩家或被玩家攻击时，会进入战斗状态并刷新 battleTime，玩家离开检测范围后，敌人不会马上停止，而是在计时结束后退出战斗状态；这样既能减少敌人全图追踪，也能避免玩家刚离开检测边缘时敌人突然静止的突兀表现。

在敌人朝向处理上，项目加入 targetOverlapEpsilon 作为坐标容差，当敌人与玩家横向距离非常接近时，就不再根据微小位置差反复翻转，而是保持当前朝向或直接进入攻击判断；这样的处理解决了玩家与敌人重叠、站在检测边界或贴身攻击时的抖动问题，使普通敌人行为更稳定。

普通敌人死亡后还要处理动画、碰撞和 AI 状态，如果只播放死亡动画而不关闭交互，敌人可能继续挡住玩家或继续触发攻击，如果直接销毁对象，又会失去死亡表现；项目在死亡流程中综合处理状态切换、碰撞关闭和动画播放，使敌人既能展示死亡反馈，又不会继续影响后续战斗流程。

普通敌人以 Enemy_Skeleton 为代表，Enemy_Skeleton 在 Awake 中创建 idleState、moveState、battleState、attackState 和 stunnedState，并在 Start 中初始化为 idleState；巡逻和待机状态用于敌人未发现玩家时的基础行为，battleState 用于发现玩家后的追踪和战斗判断，attackState 用于播放攻击动画并通过动画事件检测玩家，stunnedState 则用于受击后的短暂硬直。

敌人进入战斗状态的方式包括看到玩家、被玩家攻击以及受到黑洞伤害，Enemy_Skeleton 重写 Damage 和 TakeBlackholeDamage 方法，在调用 base 逻辑后执行 RefreshBattleTime，使敌人在被攻击后能够重新感知玩家并进入战斗状态；这样的处理解决了敌人只在主动看到玩家时才反应的问题，玩家主动攻击也能触发敌人的战斗逻辑。

为了减少敌人出现不自然抖动，项目在多个位置加入容差和时间控制，当玩家与敌人横向距离非常小时，敌人就不再频繁翻转朝向；当敌人短时间内检测不到玩家时，会经过 battleTime 后退出战斗状态，而不是立即静止或无限追踪，这样可以减少玩家站在敌人检测边界、坐标重叠或射线断点附近时出现的左右抽搐问题。

普通敌人 AI 的实现重点不在复杂策略，而在行为闭环清晰，敌人可以从待机或巡逻进入追踪，靠近玩家后攻击，受击后刷新战斗时间，生命值归零后播放死亡动画并停止交互；这样的 AI 足以支撑 Demo 中的基础战斗体验，也为 Boss 战状态设计提供了结构基础。

4.5 关卡机关与精英敌人实现

关卡机关模块主要围绕落石陷阱和骷髅王通路解锁两部分展开，落石陷阱由FallingStoneTrap负责控制，脚本内部使用Ready、Warning、Falling、Landed、Returning和Cooldown等状态描述石块生命周期；Ready表示机关可触发，Warning通过轻微抖动提示玩家危险即将发生，Falling根据加速度向下移动，Landed保持短暂停留，Returning缓慢回到初始位置，最后进入Cooldown以减少连续触发。

触发检测没有直接放在石块本体上，而是为每个石块创建独立的FallingStoneTrapSensor，传感器是触发器Collider2D，只覆盖石块正下方一定宽度区域，当玩家进入该区域时调用对应石块的Trigger方法；这样可以保证“谁检测到谁下落”，减少多个石块被同一个触发器连带触发，由于传感器固定在世界空间，石块下落和回收不会改变检测区域位置，机关重复运行时也更稳定。

石块运动使用Coroutine逐帧推进，WarningRoutine负责预警抖动，FallingRoutine通过currentFallSpeed和fallAcceleration逐渐提高下落速度，并用MoveTowards靠近底部点，LandedRoutine负责落地停留并持续检测玩家重叠，ReturningRoutine则以returnSpeed缓慢回到初始位置；和直接使用刚体重力相比，这种脚本控制方式更容易保证每次下落距离、速度上限和回收位置一致，适合毕业Demo中需要稳定复现的机关效果。

落石伤害采用接触即死处理，FallingStoneTrap在下落和落地阶段通过

damageCollider.OverlapCollider 检查玩家层级，如果检测到 Player，就关闭无敌状态并调用 TakeDamage 造成超过最大生命值的伤害；脚本中使用 killedPlayerThisDrop 标记减少同一次下落重复触发死亡，这一实现让机关危险性明确，也便于测试时判断玩家是否成功通过机关区域。

就当前 Demo 而言，骷髅王不是重新编写一套完全独立的 AI，而是在普通 Enemy_Skeleton 基础上进行场景配置和参数强化，它保留普通骷髅的待机、移动、战斗、攻击、受击和死亡状态，但拥有更高生命值、更大体型、更明显的世界空间血条，并在头顶添加金色扁平光环来区分普通敌人；这样既复用已有状态机结构，也能让玩家直观看出该敌人与普通骷髅不同。

骷髅王击败后的通路变化由 SkeletonKingTileGate 实现，该脚本持有骷髅王的 CharacterStats 引用、Ground Tilemap 引用和需要移除的 gateCells 数组，Awake 阶段缓存这些格子的瓦片、变换矩阵和颜色，OnEnable 阶段订阅 onHealthChanged 事件；当检测到骷髅王 currentHealth 归零时，OpenGate 遍历指定 Tilemap 坐标并执行 SetTile(cell, null)，由于被移除的是带碰撞的 Ground Tilemap 格子，视觉砖块消失后，TilemapCollider2D 也不会再阻挡玩家。

通路开启方案最初考虑过半透明砖块，实际演示时，这种提示容易让玩家误以为砖块仍然挡路，最终方案把 openedAlpha 设为 0，骷髅王被击败后，柱子底部三行砖块直接消失；这种反馈更直接，碰撞和显示也不会互相矛盾。

（此处替换为游戏运行截图：落石机关、骷髅王和击败后通路开启效果）

图 4.3 关卡机关与骷髅王解锁效果

表 4.4 关卡机关相关脚本职责

脚本或对象	核心职责	说明
FallingStoneTrap	控制石块预警、下落、停留、回收和冷却	通过状态枚举和 Coroutine 保证机关可重复运行
FallingStoneTrapSensor	检测玩家是否进入石块下方区域	每个传感器只触发对应石块，避免联动误触发
SkeletonKing	作为强化骷髅敌人承担关卡小 Boss	复用 Enemy_Skeleton 状态机，通过参数、血条和光环区分
SkeletonKingTileGate	监听骷髅王死亡并移除 Tilemap 砖块	通过 SetTile(null)同时取消显示和碰撞阻挡

落石机关的关键并不是让石块简单向下移动，而是让它在多次运行中保持确定性，机关每次触发时都从缓存的 startPosition 开始，向 bottomPosition 移动，结束后再回到 startPosition；如果直接依赖 Rigidbody2D 重力，石块可能受到碰撞、帧率和物理材质影响而产生位置偏差，使用 Coroutine 和 MoveTowards 虽然物理真实感稍弱，但能保证演示时每次下落距离和回收位置一致，对于答辩演示而言，稳定复现比复杂物理模拟更重要。

预警阶段的轻微抖动承担反馈提示作用，如果石块检测到玩家后立即下落，玩家会觉得伤害过于突然，加入 warningTime 后，玩家能看到石块即将运动，并有短暂时间通过冲刺离开危险区域；这个设计和 Boss 激光预警线的思路接近，二者都遵循“先提示、再造成伤害”的原则，即使玩家失败，也能理解失败原因，系统表现会比无提示的瞬间伤害更公平。

机关伤害采用即死判定是出于演示清晰度考虑，普通敌人和 Boss 攻击通常造成固定伤害，玩家可以通过血条观察损失，而落石属于场景危险物，如果只扣除少量生命值，危险感会减弱，也不容易在演示中表现机关意义；项目通过 TakeDamage 传入超过最大生命值的数值实现即死，并在触发前关闭玩家无敌状态，减少玩家刚受击后的短暂无敌抵消机关伤害，同时 killedPlayerThisDrop 保证同一次下落只处理一次死亡，避免重复播放死亡流程。

骷髅王的设计强调“基于已有敌人的强化”，不是重新创建复杂敌人，它沿用 Enemy_Skeleton 的状态机，所以待机、追踪、攻击、受击、死亡等行为和普通骷髅一致，差异主要来自数值、体型、血条和视觉标识；这样做可以减少新增敌人导致的脚本风险，也符合游戏设计中精英怪的常见做法，也就是通过较少机制变化和较明显视觉差异，让玩家意识到它比普通敌人更重要，头顶光环和较大的世界空间血条就是服务这种识别。

SkeletonKingTileGate 体现了事件驱动思路，脚本没有在每一帧主动遍历场景查找敌人是否存在，而是订阅 CharacterStats 的 onHealthChanged 事件，只在血量变化时检查是否需要开门；虽然当前项目规模较小，即使每帧检查也不会带来明显性能问题，但事件驱动更符合模块解耦思路，角色属性负责发出生命值变化通知，关卡门只关心“骷髅王是否已死亡”这一结果，不参与敌人的攻击、移动和动画细节。

Tilemap 开门时缓存原始瓦片也有维护意义，当前最终效果是直接让砖块消失，

但脚本仍保留 cachedTiles、cachedTransforms 和 cachedColors，原因是这种设计可以支持后续扩展；如果希望通路开启时先闪烁、渐隐或转移到无碰撞显示层，就能复用缓存数据，而不需要重新读取地图或手动配置素材，当前 Demo 只启用最直接的消失效果，是在可读性、演示明确性和开发成本之间做出的取舍。

就当前 Demo 而言，机关与精英怪共同补足了 Boss 前的过渡区域，玩家经过落石时需要使用移动和冲刺，击败骷髅王时需要使用攻击和技能，随后通路打开才进入 Boss 区域；这一流程使玩家在最终 Boss 前已经接触主要操作，不会出现所有系统都集中到 Boss 战才第一次使用的情况，对于论文说明来说，也让场景设计、敌人 AI、技能系统和角色属性之间有更强关联，而不是各自孤立存在。

4.6 Boss 战斗系统实现

Boss 战斗在普通敌人基础上加入了节奏控制，Boss 不会一直追着玩家做单一攻击，而是根据攻击距离、冷却时间和当前阶段选择行动；靠近玩家时，Boss 优先尝试近战，距离较远或条件满足时会释放法术，位置不合适时则通过传送调整站位，这些行为让 Boss 战比普通敌人战斗更有变化。

传送逻辑需要考虑地面和障碍物，否则 Boss 可能传送到地下或墙体内部，项目在传送位置计算时以玩家附近为候选区域，并通过向下射线寻找地面，再结合范围检测判断位置是否可用；这样既能保持 Boss 传送到玩家附近的压迫感，也能减少传

送后卡住或掉出场景的概率。

二阶段转场是 Boss 战展示性的核心部分，当 Boss 生命值低于阈值后，不会立即回到普通攻击循环，而是切换到 phaseTransitionState，通过激光预警线给玩家明确提示；预警线会先锁定玩家位置，短暂停顿后变粗并造成伤害，连续几道激光结束后再激活二阶段，这种设计让阶段变化有可见反馈，也给玩家留下通过移动或冲刺躲避的机会。

Boss 死亡流程承担 Demo 结束条件，Boss 生命值归零后设置死亡标记，停止刚体移动，关闭碰撞并播放死亡动画，随后由 UI 流程显示通关界面；通关界面给出重新开始和退出选项，使 Demo 从开始菜单、战斗过程到结束菜单形成完整闭环，对于毕业设计演示而言，这种闭环比单纯做出一个敌人更能体现系统完成度。

Boss 模块以 Enemy_DeathBringer 为核心，该类继承 Enemy，并在 Awake 中创建 idleState、battleState、attackState、spellCastState、phaseTransitionState、teleportState 和 deadState；Boss 默认处于 idleState，当玩家进入 bossFightDistance 范围后，ShouldStartBossFight 返回 true，Boss 开始进入战斗流程，相比普通敌人，Boss 状态更多、切换条件也更复杂，所以更能体现有限状态机对复杂行为的组织能力[3,10-11]。

Boss 第一阶段主要包含靠近玩家和近战攻击，为了减少 Boss 一直追到玩家身边后停住挨打的情况，项目加入攻击节奏控制，minAttackCooldown 和

maxAttackCooldown 用来随机生成攻击冷却，chaseBeforeAttackReadyTime 用来限制 Boss 只有在攻击即将可用时才更积极地追踪玩家；这样能使 Boss 在攻击间隔中保持一定停顿，减少始终贴着玩家移动造成的观感混乱。

Boss 二阶段由生命值阈值触发，当 stats.currentHealth 低于最大生命值的 phaseTwoHealthPercent 比例时，CheckPhaseTwo 将 Boss 切换到 phaseTransitionState；转场过程中 Boss 进入无敌状态并生成多道 DeathBringerPhaseLaser，激光控制器先显示细红色瞄准线，等待 warningTime 后逐渐变宽并进入伤害阶段，由于瞄准线在发射前会短暂停止追踪玩家，玩家可以根据提示通过冲刺躲避，战斗反馈比直接瞬发伤害更清楚。

二阶段激活后，Boss 通过 ActivatePhaseTwo 提高移动速度并缩短攻击冷却，同时允许施法和传送，CastSpell 根据玩家位置和移动方向生成法术预制体，TeleportNearPlayer 则调用 FindPosition 寻找合适位置；传送位置不是简单随机赋值，而是根据玩家位置、arena 范围、地面 Raycast 结果和 OverlapBox 空间检测共同决定，这样可以减少传送到地底或墙体中的情况，如果找到合法位置，Boss 会移动到玩家附近但保持 minTeleportDistanceFromPlayer 距离，使传送既有压迫感又不会直接重叠。

Boss 死亡时，Die 方法设置 isBossDead，播放 Boss 死亡音效，切换到 deadState，并冻结刚体、关闭碰撞体，PlayDeathAnimation 负责清理动画布尔值并播放 deathBringerExit 动画；Boss 生命值归零后，UI_BossClearMessage 在短暂延迟后

显示通关菜单并暂停游戏，Boss 战结束反馈更加明确，整体 Boss 系统通过阶段变化、攻击冷却、法术、传送和通关 UI 构成 Demo 中最完整的战斗流程。

Boss 开战条件同样经过了范围控制。角色只有进入 bossFightDistance 后，Boss 才会从待机进入战斗，这样玩家在远处处理普通敌人或机关时，不会提前把最终战斗拉进当前画面。进入战斗后，Boss 并非每一帧都向玩家压近，攻击冷却、施法冷却和追踪时机共同影响下一步动作。玩家因此能看到靠近、停顿、近战、施法和位移之间的节奏变化，Boss 也不会只剩下持续贴身追逐这一种表现。

二阶段激光的实现更强调提示与判定的一致性。DeathBringerPhaseLaser 使用 LineRenderer 绘制红色线段，预警时线宽较细，真正发射时再逐步增宽；激光在发射前会锁定一次方向，之后不再持续追踪玩家，玩家看到提示后仍有闪避空间。伤害检测使用与线段方向一致的 OverlapBoxAll，检测区域长度覆盖画面，宽度随激光发射阶段变化，视觉上被激光覆盖的位置和受伤范围尽量接近，这样能减少“看见打中却没扣血”或“看不清为何受伤”的落差。

传送行为处理的是另一类稳定性问题。候选位置不是单纯在玩家左右给一个偏移值，而是先根据当前可搜索区域得到目标横坐标，再向下 Raycast 查找地面，通过 Boss 碰撞体底部偏移把角色放到可站立高度，最后用 OverlapBox 判断周围是否仍被地形占据。若多次尝试后都找不到合法位置，脚本不会强行传送。保留这一退路能减少 Boss 进入墙体、掉到地底或与玩家重叠的异常情况，也让复杂战斗状态更容易

回归测试。

（此处替换为游戏运行截图：Boss 近战、法术或二阶段激光提示画面）

图 4.4 Boss 战斗与二阶段攻击效果

表 4.5 Boss 主要状态及切换条件

状态	触发条件	主要行为
Idle	玩家未进入 Boss 范围	保持待机，等待战斗开始
Battle	玩家进入范围且 Boss 未死亡	面向玩家，判断追踪、攻击、施法或传送
Attack	玩家进入攻击距离且冷却结束	播放近战攻击动画并造成伤害
SpellCast	二阶段且法术冷却结束	在玩家附近生成法术攻击
Teleport	二阶段且满足传送概率和冷却	寻找玩家附近合法地面位置并传送
PhaseTransition	生命值低于阈值	进入无敌并生成多道激光提示和攻击
Dead	生命值归零	播放死亡动画，关闭碰撞，触发通关菜单

Boss 战的节奏控制还体现在攻击冷却和追踪距离上，早期如果 Boss 持续追着玩家移动，即使攻击冷却没有结束，也会出现 Boss 贴在玩家身边来回跑动的情况，观感上像是无法正确进入攻击状态；为此项目让 Boss 在攻击冷却接近结束时再更积极地追踪玩家，并在距离较近时优先切换到 Idle 或攻击判断，这样 Boss 不会一直挤压玩家位置，也能让玩家看清 Boss 准备攻击的节奏。

二阶段激光的实现需要处理瞄准、公平性和伤害范围之间的关系，激光预警线会先朝向玩家位置，但在真正发射前短暂停止追踪，减少玩家无论怎样移动都被锁

定命中的情况；发射阶段再逐渐变粗，并使用与视觉宽度接近的碰撞判定检测玩家，这样既能保证激光看起来横穿屏幕、有足够压迫感，又能给玩家通过冲刺躲避的机会，如果判定线过细，玩家会觉得明明被画面覆盖却没有受伤，如果预警阶段也持续锁定，玩家又会觉得无法躲避，所以最终方案是在视觉和可玩性之间做折中。

Boss 传送逻辑的稳定性同样影响战斗体验，传送位置如果只根据玩家坐标做简单偏移，可能落在地面以下、墙体内部或与玩家重叠；项目通过向下 Raycast 寻找地面，并结合 OverlapBox 检测候选空间是否可站立，使 Boss 尽量出现在玩家附近的合法位置，传送失败时不强行移动，而是保持原位置继续执行其他状态，减少因为一次找点失败导致 Boss 卡死或离开战斗区域的问题。

Boss 与黑洞的交互也做了单独约束。攻击、施法和传送动作如果完全保持原状态，黑洞命中会因为传送无敌或状态占用而显得收益不足；当前实现中，Boss 被黑洞捕获时会打断这三类动作，回到原地停住的 Idle，再由公共伤害流程扣除生命值并给出受击闪白反馈。二阶段激光转场仍保留无敌保护，避免阶段演出被范围技能截断。结合试玩节奏，Boss 基础移动速度也做了小幅提高，使追踪阶段的压迫感更明显。

4.7 UI 与音效系统实现

UI 系统采用集中管理方式减少场景配置成本，开始菜单、暂停菜单、死亡菜单

和通关菜单都由 UI_BossClearMessage 统一创建或控制，脚本会检查 EventSystem 是否存在，并在需要时创建按钮和背景；这样即使场景中缺少部分 UI 对象，也能通过脚本保证基本流程可用，减少因为手动拖拽引用导致的空引用问题。

菜单与游戏时间的关系通过 Time.timeScale 控制，开始界面、暂停界面和结束界面显示时，timeScale 被设置为 0，玩家和敌人不再继续行动，恢复游戏或重新开始时再恢复为 1；需要注意的是，菜单按钮本身仍要可以响应，所以 UI 动画或输入逻辑不应依赖普通游戏时间，这样的处理能保证暂停和结束菜单具备实际控制效果。

音效系统使用 AudioManager 统一加载和播放 Resources/Audio/SFX 目录中的音频资源，攻击、受击、死亡、飞剑投掷、接剑、Boss 法术、激光和菜单点击等行为都通过枚举类型调用 PlaySFX；统一入口便于后续调整全局音量、单个音效倍率和重复播放冷却，也减少在多个脚本中分别配置 AudioSource 造成的管理混乱。

UI 系统大致包括血条显示、开始菜单、暂停菜单、死亡菜单和通关菜单，UI_HealthBar 通过绑定 CharacterStats 和 Image 组件实现血量显示，当角色生命值变化时，根据 currentHealth 和 maxHealth 计算 fillAmount；UI_WorldHealthBar 用来显示世界空间中的敌人血条，使普通敌人受击反馈更加直观。

界面提示方面，Demo 在玩家初始区域上方的背景处放置操作提示，提示内容覆盖移动、跳跃、冲刺、普通攻击、弹反、飞剑和黑洞按键，并在开局一段时间后淡出；这种放置方式不会把文字压在角色身上，也能让首次试玩者先读到核心操作。

敌方世界空间血条也根据试玩效果加宽加厚，普通骷髅、骷髅王和 Boss 受击时的生命变化更容易观察。

菜单流程由 UI_BossClearMessage 集中管理，Awake 阶段调用 EnsureEventSystem 和 EnsureMenus，确保场景中存在 EventSystem 并创建必要菜单对象，Start 阶段根据 PlayerPrefs 中的 SkipStartMenuKey 判断是否跳过开始菜单；这样重新开始时可以直接进入游戏，而不是再次停留在主菜单，运行时按 Esc 能够打开暂停菜单，再次按 Esc 或点击继续按钮恢复游戏，玩家死亡和 Boss 被击败时分别显示失败或通关菜单。

就当前 Demo 而言，暂停和结束流程主要通过 Time.timeScale 控制，当开始菜单或暂停菜单显示时，游戏时间被设置为 0，玩家、敌人和 Boss 的状态更新都会暂停，恢复游戏时再将 Time.timeScale 恢复为 1；为了减少鼠标点击开始按钮触发玩家攻击，开始游戏时还会调用玩家的 BlockActionInputFor 短时间屏蔽动作输入，这个细节虽然不大，但能提高 Demo 演示时的自然度。

（此处替换为游戏运行截图：开始菜单、暂停菜单、死亡菜单或通关菜单界面）

图 4.5 菜单与流程控制界面

音效系统由 AudioManager 管理，AudioManager 使用单例形式存在，并通过 Resources.Load 从 Audio/SFX 目录加载音频资源；玩家攻击、玩家受击、飞剑投掷、

飞剑接回、敌人死亡、Boss 法术、Boss 激光、菜单点击等操作均调用 AudioManager.PlaySFX 播放音效，音效不是核心算法，但能够增强攻击、受击和菜单操作的即时反馈，提高演示完成度。

菜单脚本不仅负责显示按钮，也在维持一局 Demo 的流程状态。开始菜单会把游戏时间暂停，按下开始后恢复 Time.timeScale；Esc 打开暂停界面时，继续按钮、重新开始和退出按钮会按当前状态决定是否显示；玩家死亡或 Boss 被击败后，界面延迟一小段真实时间再弹出，使死亡动画或 Boss 结束反馈有机会先被看到。重新开始时使用场景重载，并记录下一次跳过开始菜单，这样玩家在失败后能直接回到操作流程，不必重复点击开屏按钮。

AudioManager 把音效资源名称和枚举类型对应起来，播放方只需要说明当前属于攻击、受击、飞剑、Boss 激光还是菜单点击，不必在每个脚本中重复加载文件。管理器内部会缓存 AudioClip，并给重复声音设置极短冷却，避免同一种效果在短时间内堆叠得过于刺耳；单个调用还可以传入音量倍率，所以 Boss 激光、死亡音效等响度较高的素材能够单独压低。音效系统规模不大，但这一层统一入口使调试更省力。

5 系统测试

5.1 测试环境与测试方法

测试方法主要分为功能验证和回归验证，功能验证关注新功能是否按预期运行，拿飞剑来说，要检查能否投掷和召回，黑洞能否造成多次伤害，Boss 能否进入二阶段也属于这一类检查；回归验证则关注修改后是否影响旧流程，调整 Boss 攻击距离后，还要重新检查法术、传送和死亡菜单，本设计属于毕业 Demo，测试重点放在核心流程稳定和现场可演示上。

测试时还要同时关注 Unity 编辑器和 Windows 打包程序，编辑器便于查看 Console 错误、对象层级和组件引用，适合快速定位空引用、碰撞层级和动画参数问题；打包程序更接近最终演示环境，适合检查开始菜单、暂停菜单、退出按钮、音效播放和场景重开等流程在脱离编辑器后是否仍能运行，两类测试互相补充，可以减少答辩现场出现阻塞问题的概率。

在论文说明中，系统测试的目标是验证 Demo 主要流程是否能够稳定运行，重点检查玩家控制、普通敌人 AI、Boss 战、技能、UI 菜单、音效和 Windows 打包程序；测试环境为 Windows 系统，开发工具为 Unity 2022.3 LTS，测试方式以黑盒功能测试和开发过程中的回归测试为主，由于该项目为毕业设计 Demo，并且未做大规模并发测试或服务器压力测试，所以测试章节主要围绕单机运行流程展开。

测试过程分为编辑器测试和打包测试两个阶段，编辑器测试用来快速检查脚本逻辑、动画触发和碰撞判定，打包测试用来验证可运行程序在脱离 Unity 编辑器后是否可以正常启动、进入游戏、暂停、重开和退出；每次修改 Boss、技能或 UI 流程后，都需要重新运行关键路径，防止某个局部修复影响完整演示。

（此处替换为游戏运行截图：Windows 可执行程序启动或打包运行测试画面）

图 5.1 Windows 打包运行测试界面

关卡机关是后续测试中的新增重点，测试时需要从左右两个方向经过落石区域，重点观察传感器是否只触发玩家正下方的石块；石块下落、停留、回收和冷却流程也需要重复检查，如果一个石块带动其他石块下落，通常说明传感器范围或引用配置存在问题，这时需要回到场景中检查 FallingStoneTrapSensor 绑定。

骷髅王与通路解锁测试关注事件链是否完整，测试流程为先与骷髅王战斗，观察其血条、光环和受击反馈，当其生命值归零后，检查右侧左边柱子下方三行砖块是否消失，随后让玩家穿过原本被挡住的位置；该测试覆盖 CharacterStats 事件、SkeletonKingTileGate 脚本、Tilemap 格子移除和碰撞更新等多个环节。

5.2 功能测试

功能测试覆盖从游戏启动到 Boss 击败的完整路径，测试人员从开始菜单进入游戏后，先确认玩家不会因为点击按钮而立即攻击，随后测试移动、跳跃、冲刺、墙跳和普通攻击，再与骷髅敌人交互，观察敌人是否能发现玩家、追踪、攻击、受击

和死亡，最后进入 Boss 区域，检查 Boss 一阶段近战、法术、传送、二阶段激光、死亡和通关菜单，这一路径基本覆盖 Demo 答辩时需要展示的主要内容。

除主流程外，还需要测试若干边界情况，就玩家与敌人横坐标重叠而言，要观察敌人是否会频繁翻转；飞剑命中敌人边缘时，要观察是否会插在空气中；Boss 传送后，要检查是否会落到地面以下；激光从不同方向发射时，也要确认是否都能造成伤害；玩家死亡后是否正确暂停并显示死亡菜单，同样需要单独检查，这些问题虽然不一定影响系统架构，却会直接影响演示观感，所以被列为重点测试内容。

功能测试还需要沿着玩家真实路线进行，而不是只在场景中单独点开某个对象观察。一次完整测试应从开屏进入游戏，先确认相机跟随、角色移动和普通攻击，再经过小兵、机关和骷髅王区域，随后进入 Boss 战并覆盖死亡与通关两个分支。若只测试 Boss 脚本本身，开始菜单输入屏蔽、Tilemap 门解锁、场景重开后时间倍率恢复等问题就容易被漏掉。对当前作品而言，路径测试比孤立测试更接近最终答辩环境。

边界场景则用于检查功能在临界条件下是否还能保持合理表现。玩家从落石区域左右两侧通过时，传感器都不应提前触发过远石块；敌人和玩家横坐标接近时，敌人不应连续翻面；Boss 激光从画面两侧发射时，整段射线都要有伤害判定；Boss 传送失败时，也不应让战斗状态停滞。把这些现象记录到测试中，可以说明系统优化不是只围绕理想情况展开，而是来自实际运行中暴露出来的问题。

技能和 Boss 联动回归测试还需要检查临界时机：飞剑接回前不应提前出现冷却，黑洞结束后要在玩家落地时开始计时，Boss 处于攻击、施法或传送动作时被黑洞覆盖仍应实际扣血；同时还要观察黑洞最后一段伤害击杀普通骷髅时，死亡动画和碰撞释放是否连续执行。把这些检查放进主流程回归后，可以减少局部修复只在单一场景下有效的问题。

结合系统需求，本文选取主要功能进行测试，测试结果如表 5.1 所示。

表 5.1 系统功能测试表

测试项	测试内容	预期结果	测试结果
开始菜单	启动程序后点击开始游戏	菜单隐藏，游戏开始，玩家不立即攻击	通过
玩家移动	按方向键、跳跃键、冲刺键	角色能移动、跳跃、冲刺并播放对应动画	通过
普通攻击	玩家靠近敌人执行普通攻击	敌人扣血、受击并按正确方向后退	通过
飞剑技能	瞄准、投掷、命中、召回飞剑	轨迹点显示，飞剑可插入敌人并返回玩家	通过
黑洞技能	释放黑洞并让敌人进入范围	敌人冻结并受到多次伤害，结束后恢复	通过
普通敌人 AI	靠近、离开、攻击敌人	敌人能发现、追踪、攻击、脱战和死亡	通过
Boss 一阶段	进入 Boss 范围并战斗	Boss 启动战斗、靠近玩家并执行近战攻击	通过
Boss 二阶段	将 Boss 血量打到阈值以下	Boss 进入转场，生成激光提示和伤害判定	通过
暂停菜单	游戏中按 Esc	游戏暂停，显示继续、重开、退出按钮	通过
死亡/通关菜单	玩家死亡或 Boss 死亡	显示对应菜单并暂停游戏	通过

打包运行	运行 Windows 可执行程序	可脱离 Unity 编辑器完成主要流程	通过
落石机关	从左右方向进入石块下方检测区域	只有对应石块下落，玩家被砸中后死亡，石块随后回收	通过
骷髅王战斗	攻击强化骷髅并观察血条、体型和死亡	骷髅王具有更高血量，死亡后不再阻挡流程	通过
通路解锁	击败骷髅王后靠近右侧柱子底部	柱子底部三行砖块消失，玩家可以继续前进	通过

5.3 典型问题与优化

第五类问题是相机跟随范围和画面构图不稳定，项目使用 Cinemachine 控制主相机，直接移动 Main Camera 会被虚拟相机覆盖，所以需要调整 PlayerCamera 中的 Orthographic Size、Screen Y、Dead Zone 和 Damping 等参数；经过调试后，相机可以在玩家移动和跳跃时保持跟随，并减少画面下方露出背景色的问题。

第六类问题是音效音量不均衡，Boss 激光和死亡音效素材响度比较高，如果直接以默认倍率播放，会在演示中显得突兀；项目通过 AudioManager 的 volumeScale 参数对单个音效做倍率控制，在不修改音频源文件的情况下压低激光和死亡音效音量，使音效反馈更加统一。

在开发和测试过程中，项目出现过若干典型问题，第一类问题是空引用异常，拿飞剑预制体来说，如果缺少 Sword_Skill_Controller 组件，或者 SkillManager 没有正确获取技能组件、UI 引用没有绑定，都会导致运行时报错；针对这类问题，脚本

中增加必要的空值判断和 Debug.LogError 提示，并在 Prefab 和场景对象上补全组件引用，使错误定位更方便。

就当前 Demo 而言，第二类问题是敌人和玩家位置接近时产生抖动，普通敌人和 Boss 都会根据玩家相对位置翻转朝向，当二者横坐标几乎重合时，微小位置变化会导致朝向反复切换；为此项目加入 targetOverlapEpsilon 等容差判断，在距离过小时不再翻转朝向，并在敌人检测不到玩家后通过 battleTime 延迟脱战，减少状态频繁进入和退出。

第三类问题是 Boss 传送位置不稳定，最初如果直接根据玩家位置设置传送坐标，Boss 可能传送到地面以下或障碍物中；优化后，Boss 通过 Raycast 向下寻找地面，并使用 OverlapBox 检查候选位置是否有足够空间，再结合 arena 边界和 minTeleportDistanceFromPlayer 限制位置，使传送更接近玩家但不会与玩家或地形重叠。

第四类问题是技能视觉与判定不一致，飞剑如果碰撞框过大，会出现插在空气中的感觉，黑洞如果持续吸引敌人到中心，则可能造成敌人与玩家重叠时左右抽搐；针对飞剑，项目调小碰撞判定并使命中后关闭碰撞，针对黑洞，项目取消持续吸向中心，改为冻结敌人并按时间造成伤害，使视觉表现和判定更稳定。

第七类问题是机关触发区域过大或多个石块联动，最初如果使用较宽的检测区域，玩家还没有走到石块正下方就可能提前触发，或者一个触发器带动多个石块下

落；最终将检测区域缩小到石块宽度附近，并为每个石块单独绑定传感器，使玩家经过哪块石头下方就只触发哪块石头。

第八类问题出在通路解锁提示上，如果击败骷髅王后只让砖块半透明，玩家可能仍会判断这里不能走，如果只关闭碰撞但保留实体显示，又会造成视觉和物理不一致；最终处理方式是移除 Ground Tilemap 中的目标格子，使显示和碰撞一起消失，反馈也更清楚。

表 5.2 典型问题与处理方式

问题	原因	处理方式
空引用异常	Prefab 或场景对象缺少脚本和引用	补全组件绑定，增加空值判断和错误提示
敌人重叠抖动	横向距离过小时反复翻转朝向	加入坐标容差，距离过小时保持当前朝向
Boss 传送到地形内	传送点未检查地面和空间	通过 Raycast 找地面，OverlapBox 检查可站立空间
飞剑插空	碰撞框过大或命中后仍继续检测	调小碰撞范围，命中后停止刚体并关闭碰撞
菜单点击触发攻击	开始按钮点击和攻击输入共用鼠标事件	开始游戏后短时间屏蔽动作输入
落石提前触发或联动	传感器范围过大或多个石块共用触发器	缩小检测区域，每个石块绑定独立 FallingStoneTrapSensor
通路开启不清晰	半透明砖块容易被误认为仍然阻挡	击败骷髅王后直接移除目标 Tilemap 格子

第九类问题是新增内容之间可能产生连锁影响，拿骷髅王体型放大来说，攻击范围、受击范围和血条位置都需要重新观察；落石机关加入即死判定后，也要确认死亡菜单可以正常弹出，而不是只在敌人攻击导致死亡时才生效，因为 Unity 项目

中很多功能依赖场景对象引用，新增一个机关或敌人后，不能只测试它本身，还要沿着玩家死亡、场景重开、菜单恢复和再次触发机关等路径做回归验证。

第十类问题是视觉反馈和程序判定容易不一致，开发过程中出现过飞剑插入空气、Boss 激光视觉范围与伤害范围不一致、骷髅王通路半透明但玩家不确定能否通过等情况；处理这类问题时，不能只从代码正确性出发，还要从玩家观察角度判断，最终项目尽量让视觉与判定保持一致，飞剑碰撞框调小，激光发射宽度和伤害宽度接近，Tilemap 门开启时直接移除砖块，这些优化虽然不一定改变核心功能，却会明显影响演示观感。

第十一类问题是演示用 Demo 需要控制复杂度，随着功能增加，项目已经包含玩家状态机、普通敌人、技能、机关、精英怪、Boss 和 UI 流程，如果继续加入背包、钥匙数量、存档和多场景切换，短期内会增加大量测试压力；所以当前版本选择用骷髅王死亡直接开启 Tilemap 通路，而不是实现完整钥匙系统，这一取舍保留了“击败敌人后解锁道路”的设计意图，也减少为一个演示节点引入过重系统。

这些优化过程也说明，小型游戏 Demo 的完成度不只取决于功能数量。一个新功能加入后，往往会同时影响动画、碰撞、数值、层级、UI 和声音，例如骷髅王放大后，光环和血条位置需要重新调整；激光加粗后，预警线、伤害宽度和音效音量也要一起观察。论文若只写最终结果，会看不出这些实现工作，因此在测试与问题分析中保留关键取舍，能够更准确地反映项目开发过程。

当前版本仍保留后续扩展空间。落石机关已经具备可重复触发和单块检测的结构，若继续增加关卡，可以按场景需要调整预警时间和排列节奏；骷髅王通路解锁已经接入生命值事件，后续也可以换成钥匙、机关按钮或更完整的关卡事件管理。Boss 二阶段已经有阶段切换、预警攻击和结束菜单，再向后扩展时，可以继续补充招式或检查点，而不用推翻现有玩家控制与属性系统。

5.4 测试结果分析

从模块角度看，玩家控制、技能系统、普通敌人 AI、Boss 战和 UI 流程之间已经能够形成基本闭环，玩家可以使用移动和技能进入战斗，敌人可以对玩家行为作出反应，Boss 可以根据生命值进入二阶段，UI 可以在玩家死亡或 Boss 被击败后给出明确结果；各模块虽然仍有扩展空间，但已经可以支撑毕业设计的系统说明和现场演示。

从维护角度看，项目后续扩展方向较清晰，如果继续增加敌人类型，可以复用 Enemy、EnemyStateMachine 和 CharacterStats，如果继续增加技能，可以在 Skill 基类和 SkillManager 基础上扩展新的技能脚本，如果扩大关卡规模，可以在现有 Tilemap 和场景流程基础上增加新的区域、门锁和检查点；当前 Demo 保留了这些扩展入口，所以论文可以把其作为后续改进方向，而不必夸大当前版本的完成度。

从测试结果看，系统已经能够完成毕业 Demo 所需的主要流程，玩家可以从开

始菜单进入游戏，完成移动、跳跃、冲刺、普通攻击和技能释放，普通敌人可以产生基础 AI 反应，Boss 能够完成一阶段、二阶段和死亡流程，UI 菜单可以覆盖开始、暂停、失败和通关场景，打包程序也可以脱离 Unity 编辑器运行。

系统仍存在一定不足，一方面是关卡规模较小，尚未形成完整大型非线性地图，另一方面是背包、完整存档和复杂能力解锁系统没有作为当前 Demo 的重点实现，同时敌人类型和技能数量也较少，战斗丰富度仍有提高空间；这些不足不会影响当前毕业 Demo 的完整演示，但可以作为后续扩展方向。

总体来看，项目稳定性和可演示性已基本达到预期，玩家控制、敌人 AI、Boss 战和 UI 流程能够形成闭环，也能支撑后续论文分析。

测试结果还说明，系统的重点已经从单个脚本可运行，转到多模块衔接是否顺畅。Windows 打包程序能够启动，开始菜单能正确暂停场景，玩家进入后不会误触攻击，战斗中血条和音效会随伤害变化，失败与通关都能进入对应菜单，这些表现共同说明 Demo 已经具备基本发布形态。若只在编辑器内看某个状态切换，很难覆盖这一层流程结果。

与此同时，当前测试结论仍保持在可验证范围内。项目验证的是单机关卡流程、角色状态和交互反馈，并没有开展并发性能、联网同步或长期存档可靠性测试，所以测试分析只对现有 Demo 给出结论。把范围说清楚反而更稳妥，能够让后续改进方向和当前完成内容分开，避免论文把尚未实现的能力写成既定成果。

打包验证还承担了和编辑器检查不同的任务。编辑器能够快速查看 Console、组件引用和场景层级，适合在修改碰撞或动画后立即定位问题；可执行程序则更接近实际展示环境，需要检查开屏、退出按钮、重开流程、音量反馈和窗口运行是否正常。两类检查结合起来，既能发现开发阶段的脚本错误，也能避免答辩时只在 Unity 里能运行，离开编辑器后却出现菜单或资源加载异常。

因此，测试章节中保留关键运行截图和菜单节点会更有说服力。玩家控制画面、机关区域、Boss 二阶段提示和通关界面分别对应不同模块，截图不只是装饰，也能证明论文描述的交互已经在同一项目中跑通。后续补图时只需要截取这些真实节点，便能和正文叙述形成对应关系。

结　论

在实现过程中，项目经历了多轮问题修复、功能收敛和关卡流程补充，早期问题主要集中在技能预制体引用、碰撞判定和空引用异常，中期重点解决普通敌人与 Boss 的状态切换、检测范围和受击反馈，后期则围绕菜单流程、Boss 二阶段提示、落石机关、骷髅王通路解锁、音效平衡和 Windows 打包进行完善；通过这些调整，系统逐步从单个角色控制原型发展为包含开始、战斗和结束流程的可运行 Demo。

本课题的实践意义在于把 Unity 2D 开发中的多个常见技术点组织到同一项目中，包括 Tilemap 场景、Rigidbody2D 物理、Animator 动画、有限状态机、碰撞伤害、UI 菜单、音效加载和可运行程序发布；虽然项目规模有限，但其结构能够比较清楚地表现游戏 Demo 从需求分析、总体设计到详细实现和测试验证的基本过程。

论文以 Unity 引擎和 C#语言为基础，设计并实现了一款 2D 类恶魔城动作游戏 Demo，项目围绕毕业设计可运行、可演示和可说明的目标，完成玩家控制、普通敌人 AI、Boss 战斗、技能系统、血量显示、菜单流程、音效管理和 Windows 打包发布等功能；系统使用有限状态机组织玩家、敌人和 Boss 行为，使复杂动作切换和 AI 逻辑能够按状态拆分，提高代码结构清晰度和后续维护性。

在玩家控制部分，系统实现移动、跳跃、冲刺、墙面交互、普通攻击、飞剑和黑洞技能等功能；在敌人 AI 部分，骷髅敌人能够完成巡逻、发现玩家、追踪、攻击、

受击到死亡的流程；在关卡交互部分，落石机关能够根据玩家位置独立触发并循环回收，骷髅王被击败后能够解除 Tilemap 通路；在 Boss 设计部分，DeathBringer 具备近战攻击、法术攻击、传送、二阶段激光转场和死亡反馈；在 UI 部分，系统实现开始、暂停、死亡和通关菜单，并通过血条显示角色生命值，从测试结果看，Demo 能够在 Windows 环境下独立运行并完成主要战斗与关卡演示流程。

本轮完善后，系统的演示闭环更加接近一个完整游戏关卡，玩家不只是单纯与敌人交换伤害，还需要经过机关区域、识别精英敌人、通过战斗解锁道路，再进入最终 Boss 战；该流程虽然规模不大，但已经包含横版动作游戏常见的移动挑战、战斗挑战和流程门控，对毕业设计而言，这些内容能够说明项目不仅实现了角色控制，也考虑到关卡组织和玩家体验。

从技术收获看，项目加深了对 Unity 组件协作方式的理解，Rigidbody2D 和 Collider2D 负责物理交互，Animator 负责动作表现，Tilemap 负责地图和碰撞基础，MonoBehaviour 脚本负责把输入、状态、事件和 UI 连接起来；开发过程中出现的问题也说明，游戏项目的难点往往不在某一行算法，而在多个组件之间的引用、时序和反馈是否一致，通过反复调试空引用、碰撞范围、相机边界、音效音量和菜单时间控制，项目逐步形成了较稳定的运行状态。

后续继续完善时，可以优先从三个方向展开，一是把骷髅王通路扩展为正式钥匙或机关门系统，使不同敌人、道具和机关都能触发统一的 LevelEvent，二是补充

更多关卡截图和运行记录，让论文中的图 4.3、图 4.4 和图 4.5 都来自真实场景，三是进一步平衡敌人伤害、攻击冷却和玩家技能冷却，使 Demo 不只是能够通过，也能在体验上更接近正式动作游戏。

受开发周期限制，本项目仍有不足，关卡规模偏小，还没有完整的大型地图和复杂能力门锁，存档和背包系统只作为后续方向保留，当前 Demo 没有把这些内容完整展开；敌人类型和技能数量也不算多，战斗平衡还有继续打磨的空间，后续可以在现有架构上增加地图区域、能力解锁、道具收集、更多敌人类型和正式存档系统，使项目从毕业 Demo 继续扩展为更完整的 2D 动作游戏。

从测试覆盖角度看，当前 Demo 的验证重点已经从单个功能是否存在，转向多个功能能否连续运行，玩家可以先经过普通敌人，再躲避落石机关，击败骷髅王打开通路，最后进入 Boss 区域并完成通关；这一条路径覆盖输入、物理、动画、AI、Tilemap 事件、UI 菜单和音效等多个模块，如果该路径能够稳定运行，说明项目已经具备较完整的毕业演示价值。

从代码维护角度看，新增机关没有破坏原有玩家、敌人和 Boss 脚本，而是放在 Scripts/Traps 目录中独立维护，落石只通过 Player 和 CharacterStats 造成结果，骷髅王开门只监听 CharacterStats 并操作 Tilemap，二者都没有直接修改玩家状态机或 Boss 状态机；这种边界有利于后续删除、替换或扩展机关内容，也符合模块化设计要求。

从作品完整度角度看，新增内容增强了 Demo 的中段流程，原本玩家可能在普通敌人之后直接进入 Boss 战，流程偏短，加入机关和精英敌人后，玩家在到达 Boss 前需要完成一段可展示的关卡挑战；这样一来，论文中的需求分析、总体设计、详细实现和测试章节都有更多实际对象可以支撑，能够避免论文只围绕主角和 Boss 两个模块展开。

对于答辩展示而言，系统说明还需要能对应到现场操作，当前 Demo 可以按照固定路线进行演示：一开始从开始菜单进入游戏，展示玩家移动、跳跃、冲刺和普通攻击，随后与普通骷髅交互，说明敌人检测、追踪、攻击和死亡流程，接着通过落石机关，说明触发器、协程和即死判定，击败骷髅王后展示 Tilemap 通路开启，最后进入 Boss 战，展示一阶段攻击、传送、法术、二阶段激光和通关菜单；这条演示路线与论文结构基本一致，可以把抽象模块落到具体画面中。

从代码阅读和后续维护角度看，项目仍保持了相对清楚的目录边界，玩家行为集中在 Player 目录，敌人和 Boss 集中在 Enemy 目录，技能集中在 Skills 目录，机关集中在 Traps 目录，UI 和音效分别由 UI 与 Managers 目录承担；新增落石和骷髅王通路后，机关逻辑没有被写入 Player 或 Boss 脚本，而是通过少量引用与事件完成协作，这种组织方式有助于在答辩时说明模块职责，也便于后续发现机关问题时快速定位到对应脚本。

如果后续继续推进为更完整的课程作品或个人作品，可以优先把当前已经验证

过的机制抽象为可复用配置，落石机关可以扩展为统一 Trap 基类，支持尖刺、移动平台和地刺等不同危险物，骷髅王开门可以扩展为门锁组件，由敌人死亡、钥匙拾取或机关按钮触发，Boss 激光也可以扩展为独立攻击预制体，供其他 Boss 复用；这样既能保留当前 Demo 已经调通的稳定逻辑，也能让项目在后续扩展时减少重复编写相似代码。

参 考 文 献

[1] Bhosale T, Kulkarni S, Patankar S N. 2D Platformer Game in Unity Engine[J]. International Research Journal of Engineering and Technology, 2018, 5(4): 3021-3024.

[2] Pichlmair M, Fasterholdt M, Holmgård C. You say jump, I say how high- Operationalising the game feel of jumping[C]//Proceedings of DiGRA/FDG. 2016.

[3] Millington I. Artificial Intelligence for Games[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[4] Gregory J. 游戏引擎架构[M]. 叶劲峰, 等译. 北京: 电子工业出版社, 2014.

[5] Nystrom R. 游戏编程模式[M]. 侯金标, 译. 北京: 人民邮电出版社, 2015.

[6] Swink S. Game Feel: A Game Designer's Guide to Virtual Sensation[M]. Burlington: Morgan Kaufmann, 2008.

[7] Schell J. The Art of Game Design: A Book of Lenses[M]. 3rd ed. Boca Raton: CRC Press, 2019.

[8] Unity Technologies. Unity User Manual 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/Manual/index.html.

[9] Unity Technologies. Unity Scripting API 2022.3 LTS[EB/OL]. [2026-05-13]. https://docs.unity3d.com/2022.3/Documentation/ScriptReference/.

[10] Bourg D M, Seemann G. AI for Game Developers[M]. Sebastopol: O'Reilly Media, 2004.

[11] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2004.

[12] Rabin S. Introduction to Game Development[M]. 2nd ed. Boston: Cengage Learning, 2010.

[13] Adams E. Fundamentals of Game Design[M]. 3rd ed. Berkeley: New Riders, 2014.

[14] Trefry G. Casual Game Design: Designing Play for the Gamer in All of Us[M]. Burlington: Morgan Kaufmann, 2010.

[15] Microsoft. C# documentation[EB/OL]. [2026-05-13]. https://learn.microsoft.com/dotnet/csharp/.

致　　谢

就当前 Demo 而言，本次毕业设计从选题、功能实现到论文撰写，都得到了指导教师的耐心指导和帮助，在项目开发过程中，老师对课题方向、系统功能边界和论文结构提出了重要建议，使我能够把 Unity 项目开发与毕业论文写作结合起来，逐步完成从 Demo 实现到文档整理的工作。

同时，感谢在学习和开发过程中给予帮助的同学和朋友，在调试玩家控制、敌人 AI、Boss 战和 UI 菜单时，他们的试玩反馈帮助我发现了许多细节问题，也促使我不断完善 Demo 的稳定性和演示效果。

还要感谢大学期间所有授课教师的培养，通过本次毕业设计，我对面向对象程序设计、Unity 组件化开发、有限状态机、游戏交互和项目版本管理有了更完整的认识，这些经历也会成为今后继续学习和实践的重要基础。